



Object Oriented Software Development

Basic GUI & Event Handling



Agenda

▶ Agenda

- ▶ Introduction to Basic GUI
- ▶ Introduction to Basic Event Handling

▶ Reading

- ▶ Java how to program Late objects version (D & D)
 - ▶ Chapter 12
- ▶ The Java Tutorial:
 - ▶ Introduction to Event Listeners
 - <http://docs.oracle.com/javase/tutorial/uiswing/events/intro.html>



1. Basic GUI Programming Concepts

- ▶ A graphical user interface (GUI) presents a user-friendly mechanism for interacting with an application.

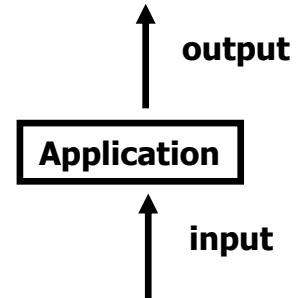
- ▶ Pronounced “GOO-ee”
- ▶ Gives an application a distinctive “look-and-feel.”

- ▶ Conventional programming:

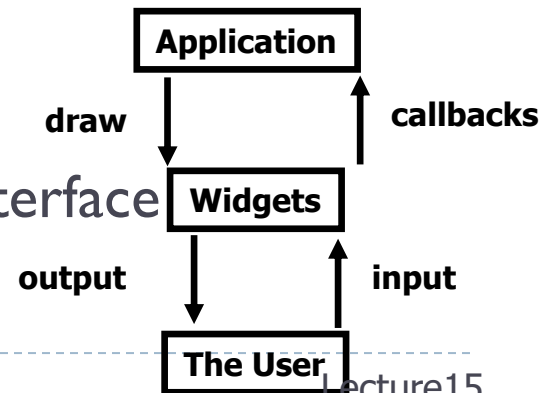
- ▶ sequence of operations is determined by the program
- ▶ what you want to happen, happens when you want it

- ▶ Event-driven programming:

- ▶ sequence of operations is determined by the user’s interaction with the application’s interface
- ▶ anything that can happen, happens at any time



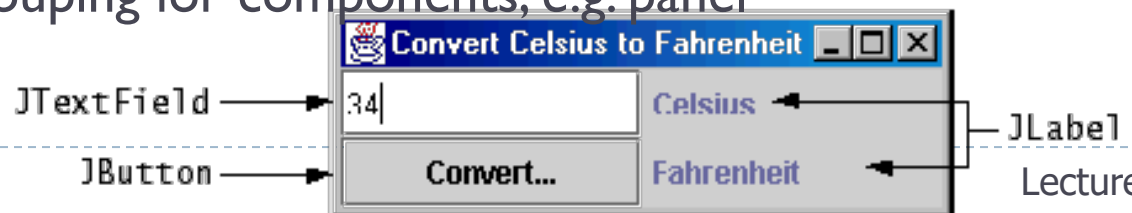
GUI-based





2. Graphical User Interface (GUI) Implementing GUIs in Java

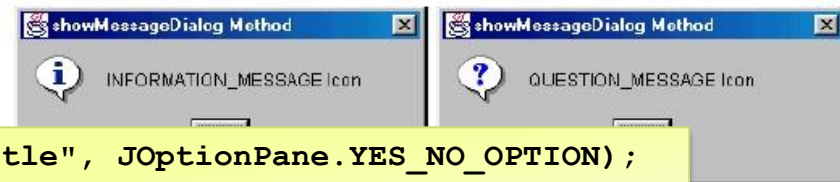
- ▶ The Java Foundation Classes (JFC) are a set of packages encompassing the following APIs:
 - ▶ AWT – Abstract Windows Toolkit (java.awt package)
 - ▶ Considered to be “heavy-weight” components using native GUI system elements
 - ▶ Event handling mechanism
 - ▶ Organize components using layout managers
 - ▶ Swing (Java 2, JDK 1.2+) (javax.swing package)
 - ▶ Most are considered to be “lightweight” that do not rely on the native OS
 - ▶ Swing component names begin with “J”:
- ▶ GUI elements
 - ▶ **windows:** actual first-class citizens of desktop; also called top-level containers, e.g. *frame*, *dialog box*
 - ▶ **components:** GUI widgets, e.g. button, label etc
 - ▶ **containers:** logical grouping for components, e.g. panel



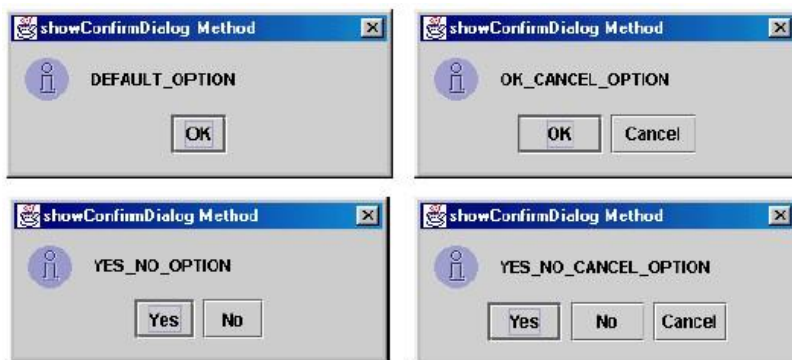


3. Simple GUI-based Input/Output Using JOptionPane

- ▶ Most applications use windows or dialog boxes (also called dialogs) to interact with the user.
- ▶ JOptionPane (package javax.swing) provides prebuilt dialog boxes for input and output (note: the user cannot interact with the rest of the application while dialog is displayed.)
- ▶ Displayed via static JOptionPane methods.



```
JOptionPane.showConfirmDialog(frame, "Message", "Title", JOptionPane.YES_NO_OPTION);
```



```
JOptionPane.showMessageDialog(frame, "Message", "Title", JOptionPane.ERROR_MESSAGE);
```



3. Simple GUI-based Input/Output showInputDialog

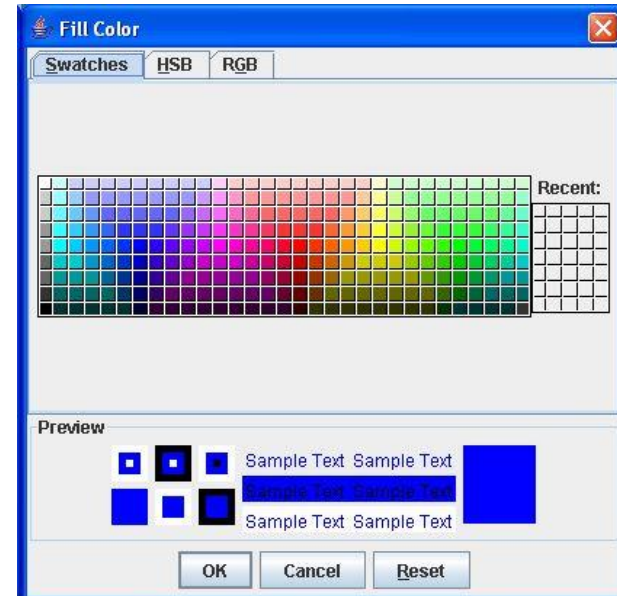
- ▶ **showInputDialog** static method
 - ▶ displays an input dialog, using the method's String argument as a prompt.
 - ▶ The user types characters in the text field, then clicks **OK** or presses the Enter key to submit the String to the program.
 - ▶ Can input only Strings. (Typical of most GUI components)
 - ▶ If the user clicks **Cancel**, returns null.
- ▶ **showMessageDialog** static method
 - ▶ displays a message dialog.
 - ▶ The first argument helps determine **where** to position the dialog.
 - ▶ If null, the dialog box is displayed at the center of your screen.
 - ▶ The second argument is the **message** to display.
 - ▶ The third argument is the String that should appear in the **title** bar at the top of the dialog.
 - ▶ The fourth argument is the **type** of message dialog to display.
 - ▶ **PLAIN_MESSAGE** dialog does **not** display an icon to the left of the message.



3.Simple GUI-based Input/Output

JColorChooser

- ▶ JColorChooser provides a pane of controls designed to allow a user to manipulate and select a color.
 - ▶ showDialog static method:
 - ▶ Arguments: parent component, title string, initially selected colour
 - ▶ The user selects a colour and clicks OK submit the selected colour to the program.
 - ▶ If the user clicks Cancel, returns null.



```
Color newColor = JColorChooser.showDialog(null, "Fill Color", Color.blue);
JOptionPane.showMessageDialog(null, "The sum is " + newColor,
    "Selected Color", JOptionPane.PLAIN_MESSAGE);
```

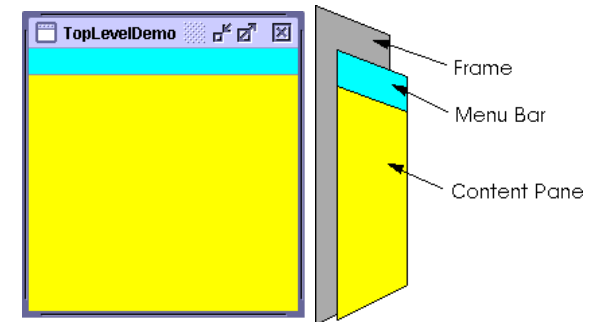


4.JFrame, JPanel & JLabel

JFrame - is a Window, a Container

- ▶ A frame is a graphical window that can be used to hold other components.
 - ▶ A JFrame object is a window with a border and a title bar.
 - ▶ The visible area of a JFrame object is a Container which means that we will be able to add components, such as a JPanel object, to the JFrame object.
 - ▶ Note: components are added to the top of a lightweight AWT Container – it is called the content pane.
 - ▶ Most windows that can contain Swing GUI components are instances of class JFrame or a subclass of JFrame.

```
getContentPane().add(emptyLabel);
```

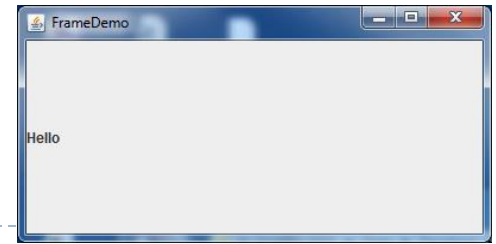


- ▶ JLabel:
 - ▶ A text label is simply a string of text displayed on screen in a graphical program. Labels often give information or describe other components



4.JFrame, JPanel & JLabel

JFrame



► Four Important Steps:

- Adding components:
- `setDefaultCloseOperation`:
 - indicates that the program should terminate when the window is closed by the user.
- `setSize/pack`:
 - `setSize`: specifies the width and height of the window in pixels.
 - `Pack`: resize the frame to fit the components inside it.
- `setVisible`:
 - with the argument `true` displays the window on the screen.

```
JFrame frame = new JFrame();  
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
JLabel emptyLabel = new JLabel("Hello");  
frame.getContentPane().add(emptyLabel);  
frame.setSize(400, 200);  
frame.setVisible(true);
```

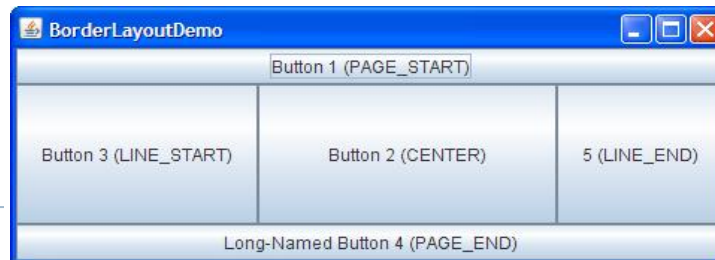
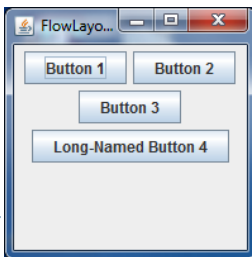


4.JFrame, JPanel & JLabel

JPanel

```
JPanel p = new JPanel();  
p.add(new JButton("OK"));
```

- ▶ You can use “new JPanel()” to create a panel for grouping components into a container, which can then be added to another container
 - ▶ Use the add(Component) method to add a component to the panel.
- ▶ It is also a dedicated drawing area
 - ▶ Override this method to specify how to draw.
- ▶ Using Layout Managers
 - ▶ Layout managers arrange GUI components in a container for presentation purposes
 - ▶ Get a layout specification as input
 - ▶ Recalculate the positions and sizes of the child widgets after each container resizing
 - ▶ Support for resizing, different screens, possibly even different devices



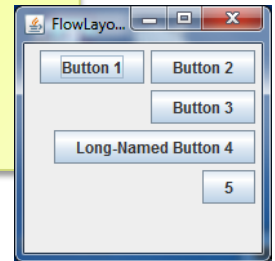


4.JFrame, JPanel & JLabel FlowLayout & BorderLayout

- ▶ Select the layout manager by setting the layout property
- ▶ Specify layout with extra parameters when adding widgets to your GUI, e.g.

- ▶ Example: FlowLayout

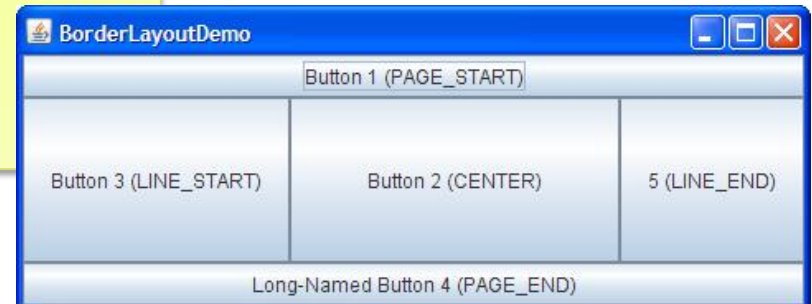
```
Container p = getContentPane();  
p.setLayout(new FlowLayout(FlowLayout.RIGHT));  
  
p.add(new JButton("Button 1"));  
p.add(new JButton("Button 2"));  
...
```



- ▶ Example: BorderLayout

- ▶ `add(component, BorderLayout.CENTER) //preferred`

```
Container pane = getContentPane();  
pane.setLayout(new BorderLayout());  
...  
pane.add(button, BorderLayout.CENTER);  
...
```





5. Custom Painting

- ▶ Create an area for custom painting/drawing inside a JPanel
- ▶ Override the paintComponent method

```
public void paintComponent(Graphics g) {  
    super.paintComponent(g);  
}
```

- ▶ Note: Call the superclass version of paintComponent as the first statement in the body of the overridden method to ensure that the component displays correctly.
- ▶ Note: We don't make a direct call to the paintComponent() method in our code.
 - ▶ This method is called **automatically** by the Java runtime whenever the JPanel area needs to be refreshed e.g.
 - ▶ when the JFrame is first created and displayed,
 - ▶ on some platforms the JPanel area is covered (the user moves to another application) and comes back to the JFrame,
 - ▶ when the user makes a change to the JFrame size.



5. Custom Painting Graphics & Graphics2D

```
public void paintComponent(final Graphics g) { ...  
    final Graphics2D g2d = (Graphics2D) g; // Just cast it...  
    // Use g2d  
}
```

- ▶ Old graphics context: `java.awt.Graphics`
- ▶ New graphics context: `java.awt.Graphics2D`
 - ▶ Although `paintComponent()` takes a `Graphics` object, what you get is really a `Graphics2D`!
- ▶ Basic methods for painting (`Graphics` and `Graphics2D`):
 - ▶ `drawLine()`
 - ▶ `clearRect()`, `drawRect()`, `draw3DRect()`, `fillRect()`, `fill3DRect()`
 - ▶ `drawArc()`, `fillArc()`, `drawOval()`, `fillOval()`
 - ▶ `drawPolygon()`, `fillPolygon()`, `drawPolyLine()`
 - ▶ `drawString()`

```
drawLine(int x1, int y1, int x2 , int y2)
```

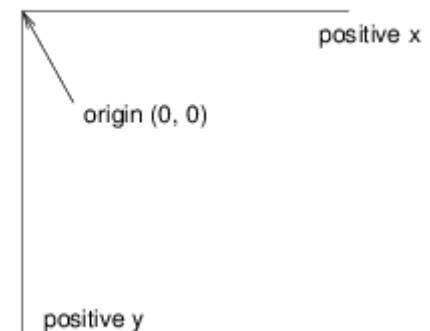
```
drawRect(int x, int y, int width, int height)
```

```
drawOval(int x, int y, int width, int height)
```

```
fillRect(int x, int y, int width, int height)
```

```
drawString(String text, int x, int y)
```

Java graphical coordinates





5. Custom Painting Java 2D

- ▶ Support for arbitrary shapes
 - ▶ A single draw() method, a single fill()
 - ▶ Draws or fills anything implementing
 - ▶ Line2D, Rectangle2D, RoundRectangle2D
 - ▶ Arc2D, Ellipse2D
 - ▶ QuadCurve2D, CubicCurve2D
 - ▶ ...
- ▶ Pen styles implement the Stroke interface (BasicStroke)
 - ▶ Different line widths, patterns, join styles
 - ▶ Use setStroke()
- ▶ Fill patterns implement the Paint interface
 - ▶ Color: Solid fill, default color space sRGB (rgb + alpha)
 - ▶ Color.RED, Color.GREEN, Color.BLACK, ...

```
Color cyan2 = new Color(0, 255, 255); // Between 0 and 255
```

- ▶ TexturePaint: Tiles a picture (repeats as necessary)
 - ▶ GradientPaint: A gradient between two colors
- ▶ Use setPaint() or the older setColor()



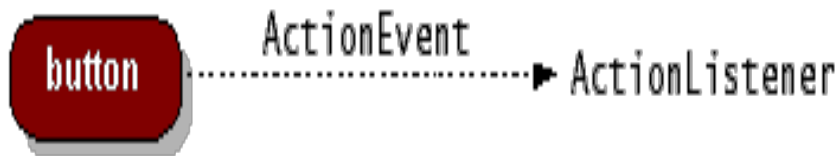
6.Event Handling

- ▶ GUIs are **event** driven.
- ▶ When the user interacts with a GUI component, the interaction—known as an **event**—drives the program to perform a task.
- ▶ The code that performs a task in response to an event is called an **event handler**, and the process of responding to events is known as **event handling**.
- ▶ On-screen components cause events to occur when they are clicked / interacted with
 - ▶ Program's execution is indeterminate



6.Event Handling Definition

- ▶ **Event source:**
 - ▶ a GUI component that generates / fires an event
- ▶ **Event:**
 - ▶ a user interaction (e.g. a click on the button)
- ▶ **Event listener:**
 - ▶ An object that has implemented event handlers to react to an event
 - ▶ **Note:**
 - ▶ the delegation event model - event listeners must be **registered** with an event source in order to receive **notification**



- ▶ Listener's appropriate method will be called when event occurs (e.g. when the button is clicked)



6.Event Handling

How Event Handling Works

- ▶ Each event type has one or more corresponding event-listener interfaces.
 - ▶ ActionEvents are handled by ActionListeners
 - ▶ MouseEvents are handled by MouseListeners and MouseMotionListeners
- ▶ When the user presses Enter in a JTextField, an ActionEvent occurs.
 - ▶ Processed by an object that implements the interface **ActionListener** .
 - ▶ To handle ActionEvents, a class must **implement** interface ActionListener and declare method actionPerformed.
 - ▶ This method specifies the tasks to perform when an ActionEvent occurs.



7.Event Handling with Inner Classes

- ▶ Before an application can respond to an event for a particular GUI component, you must perform several coding steps:

1. Create a **class** that represents the event handler.

```
class MyListener implements ActionListener {
```

Inner class

2. Implement an appropriate interface, known as an **event-listener** interface, in the class from Step 1.

```
public void actionPerformed(ActionEvent e) {  
    ...  
}
```

3. Create an instance of that class (i.e. an event listener)

```
MyListener handler = new MyListener();
```

4. Register the listener with the GUI component(s). This is known as **registering the event handler**.

1. If you forget to register for a particular GUI's component's event type, events of that type will be ignored.

```
textField1.addActionListener( handler );  
textField2.addActionListener( handler );
```

Must register for
each text field.



7. Event handling JButton & JLabel

▶ JButton:

- ▶ The most common component — a button is a clickable onscreen region that the user interacts with to perform a single command

▶ JLabel:

- ▶ A text label is simply a string of text displayed on screen in a graphical program. Labels often give information or describe other components

▶ Methods:

- ▶ **public JButton(String text)** OR **public JLabel(String text)**
Creates a new button / label with the given string as its text.
- ▶ **public String getText()**
Returns the text showing on the button / label.
- ▶ **public void setText(String text)**
Sets button / label's text to be the given string.



7.Event Handling

JTextField & JTextArea



Verify that the RJ45 cable is connected to the WAN plug on the back of the Pipeline unit.

- ▶ **JTextField:**
 - ▶ A text field is like a label, except that the text in it can be edited and modified by the user. Text fields are commonly used for user input, where the user types information in the field and the program reads it
- ▶ **JTextArea:**
 - ▶ A text area is a multi-line text field
- ▶ **Methods:**
 - ▶ **public JTextField(int columns)**
 - ▶ **public JTextArea(int lines, int columns)**
Creates a new text field that is the given number of columns (letters) wide.
 - ▶ **public String getText()**
Returns the text currently in the field.
 - ▶ **public void setText(String text)**
Sets field's text to be the given string.



7.Event Handling

Example: SimpleDemo

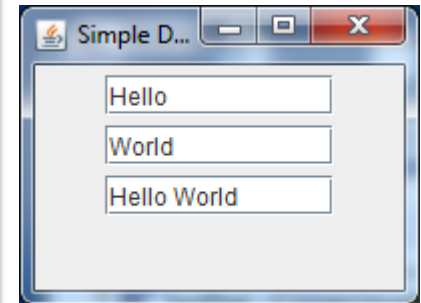
- Whenever the user presses 'Enter' in either of the first two JTextFields the third JTextField displays a String combining the shorter of the two words from the first JTextFields followed by the longer.

```
public class SimpleDemo extends JFrame {
    private JTextField textField1, textField2, textField3;
    public SimpleDemo() {
        ...
        MyListener handler = new MyListener();
        textField1.addActionListener( handler );
        textField2.addActionListener( handler );
    }
    class MyListener implements ActionListener {
        @Override
        public void actionPerformed((ActionEvent e) {
            String word1 = textField1.getText();
            ...
            textField3.setText(result);
            ...
        }
    }
}
```

new J Text field ("hello")

*create
an object*

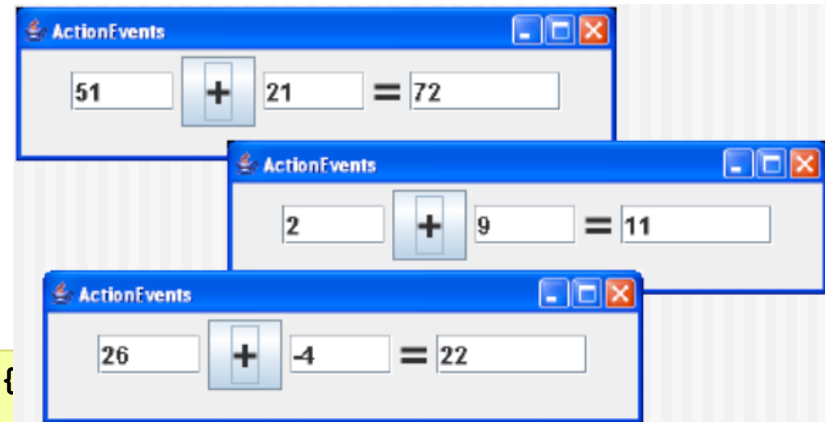
*member class
(nested)*





Exercise 1

- ▶ The user enters integers into the first two TextFields, presses the '+' JButton and the result is displayed in the third JTextField.
- ▶ Complete the inner member class



```
public class SimpleCalculator extends JFrame {  
    ...  
    addButton.addActionListener(_____);  
}
```

```
class MyListener implements ActionListener {  
    public void actionPerformed(ActionEvent e) {  
        int num1 = Integer.parseInt(textField1.getText());  
        int num2 = Integer.parseInt(textField2.getText());  
  
    }  
}
```



8.Event Handling with Anonymous

- ▶ If you have a number of buttons that do almost the same thing. You can create one listener for all the keys.
- ▶ However, there is no need to define a named class simply to add a listener object to a button.
- ▶ Note: The compiler creates names for anonymous classes. (the enclosing class name followed by \$ followed by a number, e.g., TextFieldDemo\$1.class file

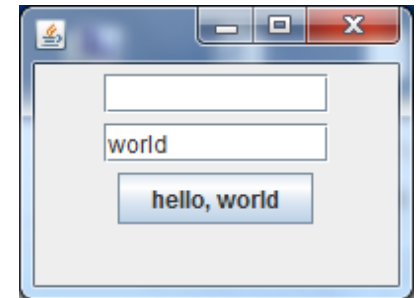
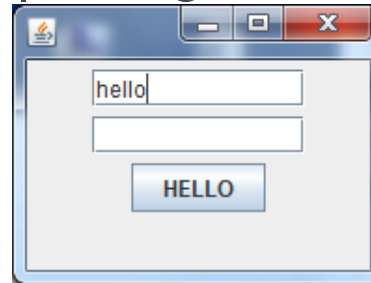
```
public class TextFieldDemo1 extends JFrame {  
    private JTextField textField1, textField2;  
    public TextFieldDemo1() {  
        ...  
        textField1.addActionListener( new ActionListener() {  
            @Override  
            public void actionPerformed((ActionEvent e) {  
                showButton.setText(textField1.getText().toUpperCase());  
            }  
        });  
    }  
}
```



8.Event Handling with Anonymous Example

▶ Event Handling

- ▶ The user presses 'Enter' in the first JTextField, the JButton displays a the corresponding text in upper case.
- ▶ The user presses 'Enter' in the second JTextField, the JButton displays “hello,” and followed by the corresponding text.



▶ Two different event handling procedures inside the actionPerformed method =>

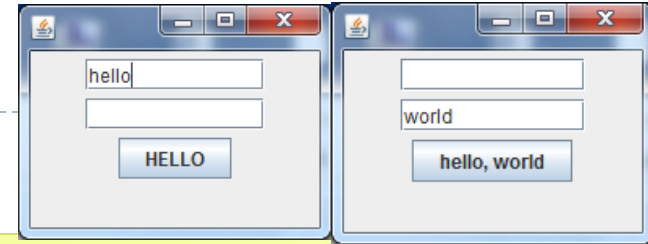
- ▶ Case 1: Use two anonymous classes to handle the event separately.
- ▶ Case 2: Using an inner member class:
 - ▶ use the getSource() to return a reference to the event source (textfield1 or textfield2), or



8.Event Handling

Case 1

► Two anonymous classes



```
public class TextFieldDemo1 extends JFrame {
    JTextField textField1, textField2;
    public TextFieldDemo1() {
        textField1 = new JTextField(10);
        textField2 = new JTextField(10);

        JButton showButton = new JButton("OK");
        textField1.addActionListener( new ActionListener() {
            @Override
            public void actionPerformed((ActionEvent e) {
                showButton.setText(textField1.getText().toUpperCase());
            }
        });
        textField2.addActionListener( new ActionListener() {
            @Override
            public void actionPerformed((ActionEvent e) {
                showButton.setText("hello, " + textField2.getText());
            }
        });
    }
}
```



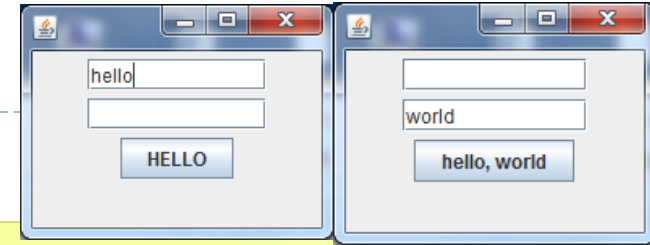
8.Event Handling

Case 2

► An inner member class

```
public class TextFieldDemo2 extends JFrame {
    JTextField textField1, textField2;
    JButton showButton;
    public TextFieldDemo2() {
        textField1 = new JTextField(10);
        textField2 = new JTextField(10);
        showButton = new JButton("OK");
        textField1.addActionListener( new MyListener());
        textField2.addActionListener( new MyListener());
        ...

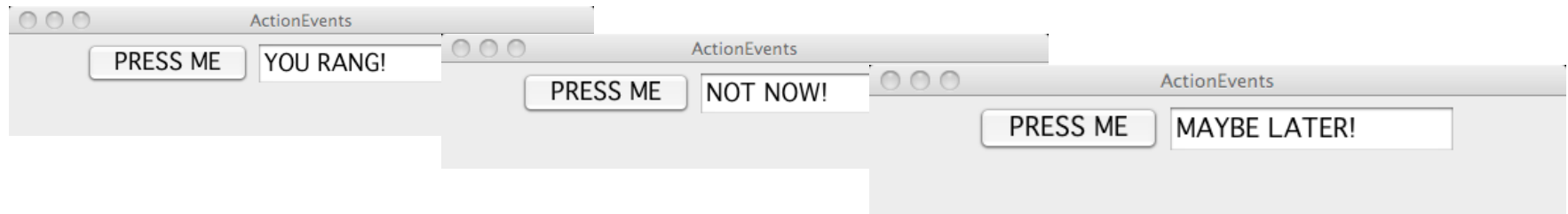
    class MyListener implements ActionListener {
        public void actionPerformed((ActionEvent e) {
            if (e.getSource() == textField1)
                showButton.setText(textField1.getText().toUpperCase());
            else if (e.getSource() == textField2)
                showButton.setText("hello, " + textField2.getText());
        }
    }
    ...
}
```





Exercise 2

- ▶ This JPanel contains a JButton and a JTextField which initially is empty.
 - ▶ The first time the user presses the 'PRESS ME' JButton the JTextField displays "YOU RANG!",
 - ▶ the second time the user presses the 'PRESS ME' JButton the JTextField displays "NOT NOW!" and
 - ▶ the third time "MAYBE LATER!" is displayed, then "YOU RANG!" again and so on.
- ▶ Some JPanel screenshots:

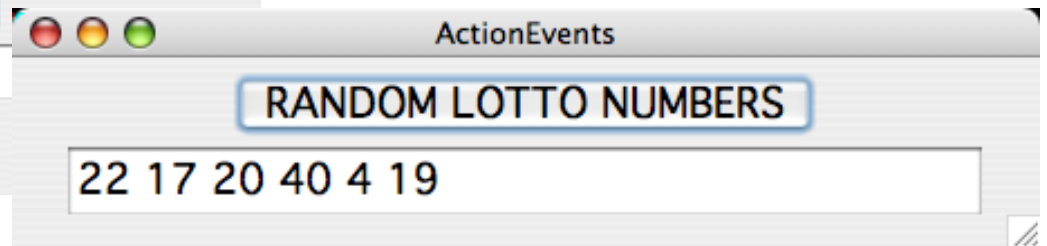
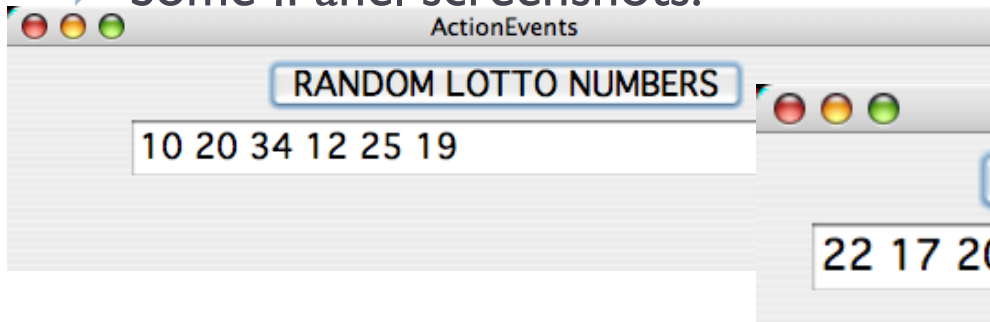


```
class MyListener implements ActionListener {  
    public void actionPerformed( ActionEvent e ) {  
  
  
  
  
  
  
    }  
}
```



Exercise 3

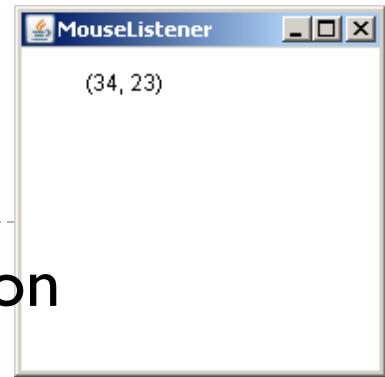
- ▶ This JPanel contains one JTextField and one JButton.
 - ▶ When the user presses the RANDOM LOTTO NUMBERS JButton six random numbers between 1 and 40 (inclusive) are displayed in the JTextField (repeats are allowed).
 - ▶ Note: use the following code to generate a specific sequence of random numbers:
 - ▶ Use `Random rand = new Random(30);`
 - ▶ Some JPanel screenshots:



```
class MyListener implements ActionListener {  
    public void actionPerformed((ActionEvent e) {  
  
    }  
}
```



Example: MouseListener



- ▶ Display (x,y) coordinate at the mouse click position
 - ▶ Case 1: Implements the MouseListener interface
 - ▶ addMouseListener
 - ▶ Implements 5 methods even you don't need all
 - ☐ mousePressed
 - ☐ mouseReleased
 - ☐ mouseClicked
 - ☐ mouseEntered
 - ☐ mouseExited
 - ▶ Invokes the repaint() method to update the position
 - ▶ Case 2: extends MouseAdapter
 - ▶ addMouseListener
 - ▶ Implements the mouseClicked method
 - ▶ Invokes the repaint() method



MouseDemo

► Using MouseAdapter

```
public class MouseDemo extends JFrame {
    int mouseX, mouseY;
    DrawPanel drawPanel;
    public MouseDemo() {
        drawPanel = new DrawPanel();
        drawPanel.addMouseListener(new MyMouseAdapter());
        ...
    }

    class MyMouseAdapter extends MouseAdapter {
        public void mousePressed( MouseEvent e ) {
            mouseX = e.getX();
            mouseY = e.getY();
            repaint();
        }
    }

    class DrawPanel extends JPanel {
        public void paintComponent(Graphics g) {
            super.paintComponent(g);
            g.drawString("(" + mouseX + ", " + mouseY + ")", mouseX, mouseY);
        }
    }
}
```